

The Linux File System

Module 8.5

- 8.5.1 The File System Types in Linux
- 8.5.2 Linux Roles and File Permissions
- 8.5.3 Hard Links and Symbolic Links

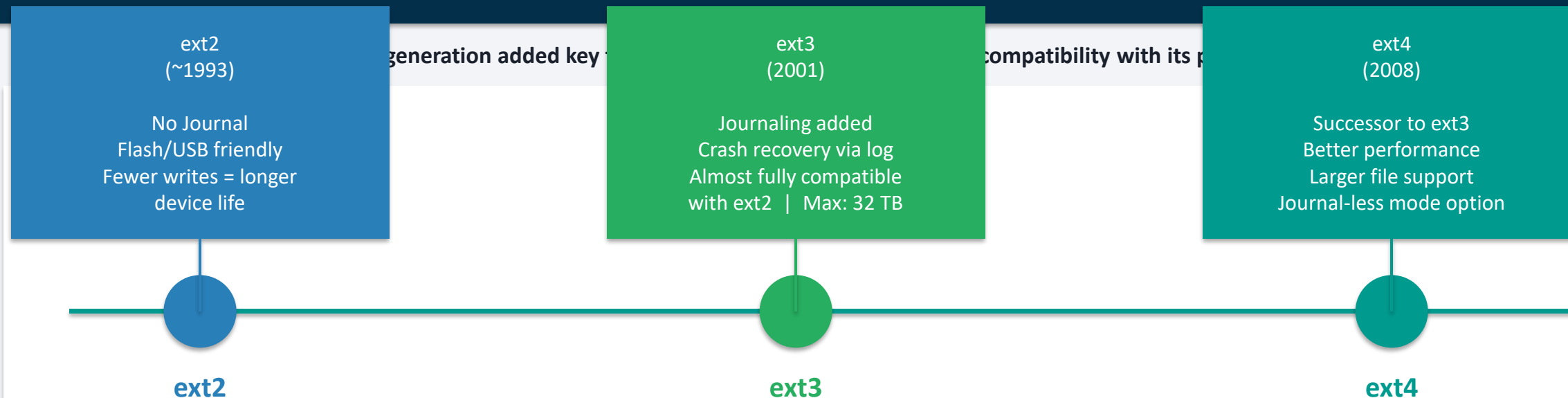
File Systems: ext2 (Second Extended)

- 1 Administrators choose file systems based on speed, flexibility, security, size, and structure needs
- 2 ext2 was the default in major Linux distributions until it was replaced by ext3
- 3 ext2 lacks a journal — this increases performance and minimizes write operations to disk
- 4 Flash-based storage (USB, SD) benefits most: limited write cycles means fewer writes = longer lifespan
- 5 Modern kernels also support ext4, which improves on ext2 and can operate in journal-less mode for flash

File Systems: ext3 and ext4

- 1 ext3 (Third Extended FS): a journaled file system designed specifically to improve ext2
- 2 Journaling logs all pending file system changes before committing them — enables crash recovery
- 3 If the computer crashes mid-operation, the journal restores or corrects any inconsistencies; max file: 32 TB
- 4 ext4 (Fourth Extended FS): created as ext3's successor with extensions for better performance
- 5 The ext3 project split into two — ext3 kept the stable base, ext4 added large-file support and speed

Visual: The Extended File System Family ext2 > ext3 > ext4



Key difference: ext2 has NO journal (good for flash). ext3/ext4 have journals (good for crash recovery on HDDs/SSDs).

File Systems: NFS, CDFS, and Swap

- 1 NFS (Network File System): allows transparent file access over a network — remote files appear local to users
- 2 NFS is an open standard — anyone can implement it; no difference between local and remote access from user view
- 3 CDFS (Compact Disc File System): created specifically for optical disk media (CDs, DVDs)
- 4 Swap file system: Linux uses this when RAM runs out — the kernel moves inactive RAM content to a swap partition
- 5 Swap is stored on disk — much slower than RAM; it is a safety net, not a substitute for physical memory

HFS+, APFS, MBR, and Mounting

- 1 HFS+ (Hierarchical File System Plus): Apple's macOS file system; Linux kernel includes read/write support
- 2 APFS (Apple File System): updated Apple FS with strong encryption, optimized for flash and solid-state drives
- 3 MBR (Master Boot Record): located in the first sector of a disk; stores partition table info and hands control to OS loader
- 4 Mounting assigns a directory (mount point) to a partition — similar to drive letters in Windows (e.g., C:\)
- 5 After mounting, the file system on that partition is accessible through the specified directory path

Visual: The mount Command and File System Types

`mount` with no arguments lists all currently mounted file systems | Root FS is `/dev/sda1` formatted as `ext4`

```
[analyst@secOps ~]$ mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sys on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
dev on /dev type devtmpfs (rw,nosuid,relatime,size=494944k,nr_inodes=123736,mode=755)
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755)
/dev/sda1 on / type ext4 (rw,relatime) <-- ROOT file system: ext4 on /dev/sda1
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
tmpfs on /sys/fs/cgroup type tmpfs (ro,nosuid,nodev,noexec,mode=755)
cgroup on /sys/fs/cgroup/systemd type cgroup (...)
tmpfs on /tmp type tmpfs (rw,nosuid,nodev)
... (additional cgroup and kernel mounts omitted for brevity)
```

`/dev/sda1 on /`

Root file system mounted
at directory `/`

`type ext4`

File system type `ext4`
(journalled, most common)

`(rw,relatime)`

Mount options:
read-write, relative atime

`tmpfs on /tmp`

`tmpfs` = RAM-based temp
store, lost on reboot

The mount point `/` represents the root of the entire Linux file system tree — all files live here by default

Linux Roles and File Permissions

- 1 In Linux, most system entities are treated as files — permissions are built into the file system structure
- 2 Every file carries three permission sets: Owner (user), Group, and Others with Read/Write/Execute rights
- 3 Use `ls -l` to view the permission string, link count, owner, group, size, date, and file name
- 4 The first character of the mode string is the file type: `-` for regular file, `d` for directory
- 5 The root user can override ANY file permission — root access must be protected with strong passwords

Visual: Reading the ls -l Permission Output

Every `ls -l` line contains 7 fields. Example: `-rwxrw-r-- 1 analyst staff 253 May 20 12:49 space.txt`

```
[analyst@secOps ~]$ ls -l space.txt
-rwxrw-r-- 1 analyst staff 253 May 20 12:49 space.txt
  (1)      (2)      (3)      (4)      (5)      (6)      (7)
```

(1) Permissions
-rwxrw-r--

File type + Owner perms
+ Group perms + Others

(2) Hard Links
1

Number of hard links
to this inode

(3) Owner
analyst

User who owns the file

(4) Group
staff

Group associated
with this file

(5) Size
253 bytes

File size in bytes

(6) Timestamp
May 20 12:49

Date and time of
last modification

(7) File Name
space.txt

Name of the file

Detailed permission string breakdown for -rwxrw-r--

-

File type
(- = file
d = dir)

rwx

Owner (user)
Read+Write+Execute

rw-

Group (staff)
Read+Write only

r--

Others
Read only

File permissions are fundamental and cannot be bypassed — only root can override them on any file

Visual: Binary, Octal and Permission Reference Table

Each permission triplet (rwx) maps directly to a 3-bit binary value and a single octal digit 0 through 7

Binary	Octal	Permission	Description
000	0	- - -	No access
001	1	- - x	Execute only
010	2	- w -	Write only
011	3	- w x	Write and execute
100	4	r - -	Read only
101	5	r - x	Read and execute
110	6	r w -	Read and write
111	7	r w x	Read, write, and execute

Each permission set = 3 bits:

r (read) = 4 (binary 100)
 w (write) = 2 (binary 010)
 x (execute) = 1 (binary 001)
 - (none) = 0 (no bit set)

Examples:

$rw x = 4 + 2 + 1 = 7$
 $rw - = 4 + 2 + 0 = 6$
 $r - x = 4 + 0 + 1 = 5$
 $r - - = 4 + 0 + 0 = 4$
 $- - - = 0 + 0 + 0 = 0$

chmod 754 file means:

Owner $rw x$ (7) = full access
 Group $r - x$ (5) = read + execute
 Others $r - -$ (4) = read only

Hard Links

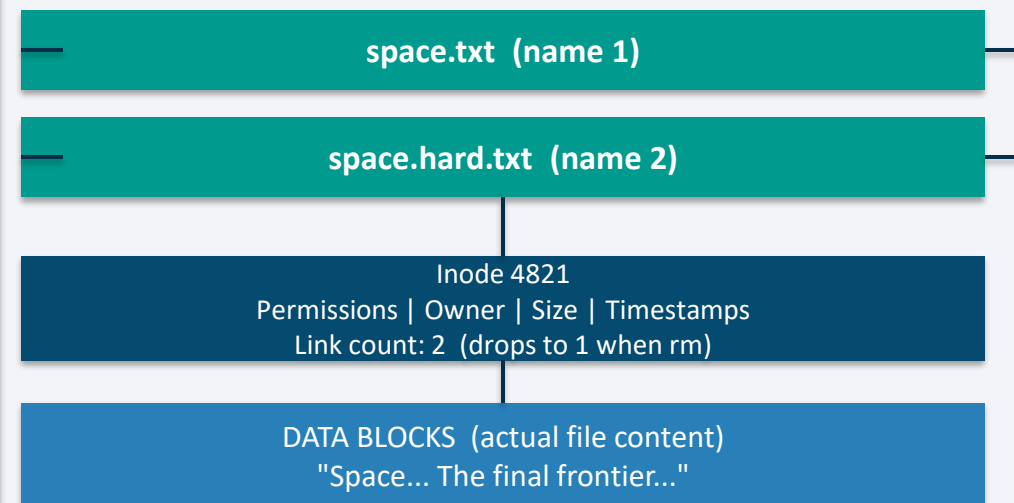
- 1 A hard link is another file name pointing to the same inode (disk location) as the original file
- 2 Create with: `In space.txt space.hard.txt` — both entries share the same inode and data blocks
- 3 The hard link count in `ls -l` increases to 2; both files show identical size and inode
- 4 Modifying one file changes the other immediately — they are two names for one set of data
- 5 Deleting one hard link does not delete the data; the inode persists until the link count reaches zero

Visual: Hard Links — Terminal Session and Diagram

In creates a hard link | Both names point to the same inode | rm one name keeps data intact

```
[analyst@secOps ~]$ ln space.txt space.hard.txt
[analyst@secOps ~]$ ls -l space*
-rw-r--r-- 2 analyst analyst 239 May 7 18:18 space.hard.txt
-rw-r--r-- 2 analyst analyst 239 May 7 18:18 space.txt
      ^ link count = 2 (two names, same inode)
[analyst@secOps ~]$ echo "Testing hard link" >> space.hard.txt
[analyst@secOps ~]$ ls -l space*
-rw-r--r-- 2 analyst analyst 257 ... space.hard.txt  <- size: 257
-rw-r--r-- 2 analyst analyst 257 ... space.txt      <- SAME size
[analyst@secOps ~]$ rm space.hard.txt
[analyst@secOps ~]$ more space.txt
Space... The final frontier...
Testing hard link          <- text still exists in space.txt
```

Inode Diagram:



After rm space.hard.txt:

Link count: 1 | Data still exists | space.txt still works

Both names CANNOT cross file-system boundaries

Symbolic Links (Symlinks / Soft Links)

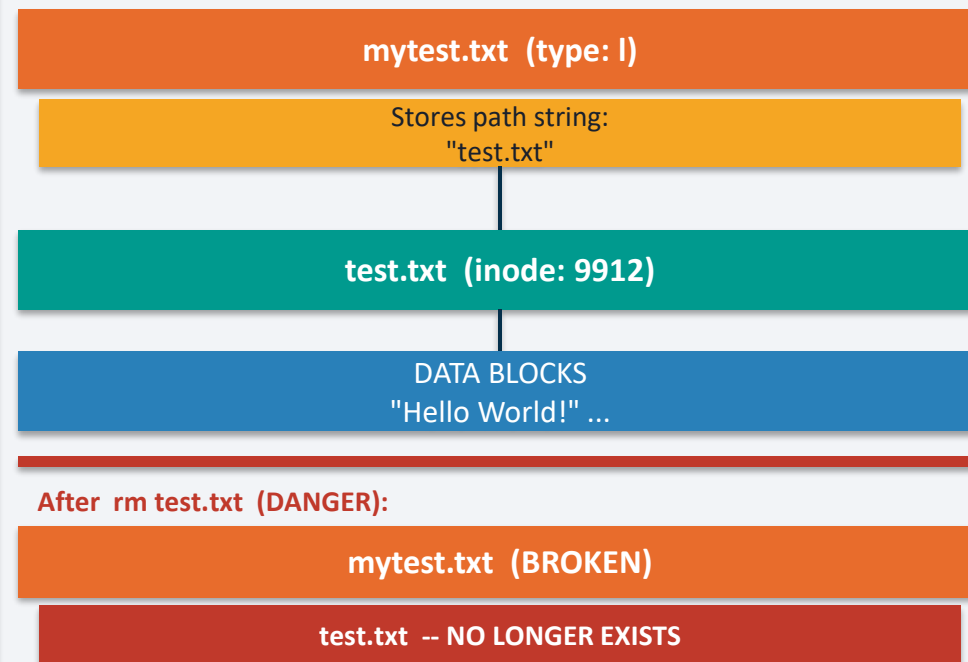
- 1 A symbolic link is a special file whose content is a PATH to the target file — like a desktop shortcut
- 2 Create with: `ln -s test.txt mytest.txt` — changes made via symlink also change the original file
- 3 Deleting the original makes the symlink “dangling” — accessing it returns a "No such file or directory" error
- 4 `ls -l` identifies symlinks with `l` as the first character and shows the arrow: `mytest.txt -> test.txt`
- 5 Advantages: easy to locate target, can cross file systems, can link to directories (unlike hard links)

Visual: Symbolic Links — Terminal Session and Diagram

In -s creates a symlink | Symlink stores a path, not data | Deleting original = dangling symlink

```
[analyst@secOps ~]$ echo "Hello World!" > test.txt
[analyst@secOps ~]$ ln -s test.txt mytest.txt
[analyst@secOps ~]$ echo "It's a lovely day!" >> mytest.txt
[analyst@secOps ~]$ more test.txt
Hello World!
It's a lovely day!      <- change via symlink appears in original
[analyst@secOps ~]$ rm test.txt
[analyst@secOps ~]$ more mytest.txt
more: stat of mytest.txt failed: No such file or directory
                        <- DANGLING SYMLINK
[analyst@secOps ~]$ ls -l mytest.txt
lrwxrwxrwx 1 analyst analyst 8 May 7 20:17 mytest.txt -> test.txt
^-- l = symlink type           ^-- arrow shows target path
```

How a Symlink Works:



Visual: Hard Link vs Symbolic Link — Side-by-Side Comparison

From the course: "Symbolic links have a single point of failure but several advantages over hard links"

Property	Hard Link	Symbolic Link
What it stores	Directory entry pointing to inode (direct)	A path string to the target file (indirect)
ls -l indicator	Normal file: -rw-r--r-- (no arrow)	l type + arrow: mytest.txt -> test.txt
Locate target easily?	No ✗ (no visible pointer shown)	Yes ✓ (ls -l always shows -> target)
Cross file systems?	No ✗ (same file system only)	Yes ✓ (can link across any file system)
Link to directories?	No ✗ (system restriction to prevent loops)	Yes ✓ (fully supported)
If original deleted?	Data survives ✓ (link count drops by 1)	Symlink breaks ✗ (dangling symlink)
Inode number	Identical to original file	Different inode from the target
Create command	ln source linkname	ln -s target linkname
Single point of failure	No ✓ (each name is equally valid)	Yes ✗ (depends on target existing)

Course note: "Although symbolic links have a single point of failure (the underlying file), they provide advantages: easy location, cross-filesystem linking, and directory linking."

- 1 Cisco Networking Academy. (2025). Endpoint Security (Module 8.5: The Linux File System). Cisco Systems. <https://www.netacad.com/courses/endpoint-security>
- 2 The Linux Kernel Organization. (2025). Filesystems in the Linux kernel. The Linux Kernel Documentation. <https://docs.kernel.org/filesystems/index.html>
- 3 GNU Project. (2025). File permissions (GNU Coreutils 9.9). Free Software Foundation. https://www.gnu.org/software/coreutils/manual/html_node/File-permissions.html
- 4 GNU Project. (2025). chmod invocation (GNU Coreutils 9.10). Free Software Foundation. https://www.gnu.org/software/coreutils/manual/html_node/chmod-invocation.html
- 5 The Linux Man Pages Project. (2025, November). ln(1) - Linux manual page (coreutils 9.9). <https://man7.org/linux/man-pages/man1/ln.1.html>
- 6 The Linux Man Pages Project. (2025, November). chmod(1) - Linux manual page (coreutils 9.9). <https://man7.org/linux/man-pages/man1/chmod.1.html>
- 7 Nguyen, B. (2004). Linux filesystem hierarchy (Version 0.65). The Linux Documentation Project. <https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/index.html>
- 8 Red Hat, Inc. (2022). Managing file systems. Red Hat Enterprise Linux 9. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_file_systems/index
- 9 Wikipedia contributors. (2026, March 13). ext4. Wikipedia. <https://en.wikipedia.org/wiki/Ext4>
- 10 Shields, I. (2016, January 27). Prepare for LPIC-1 exam 1 - topic 104.5: Manage file permissions and ownership. IBM Developer. <https://developer.ibm.com/tutorials/l-lpic1-104-5/>